

Day 1 종합실습 실행결과

데이터 수집 미니 파이프라인

작성자: 광주 3반 김지민

작성일: 2026-07-20

체크 항목	구현 및 검증 결과
환경 구성	venv + requirements.txt 설치 완료
비동기 수집	3개 API를 asyncio.gather()로 동시 호출, 모두 HTTP 200
스키마 검증	Pydantic v2 모델 3개, 총 74건 검증
저장 비교	CSV와 Parquet 저장, 읽기/쓰기 시간 및 크기 측정
테스트	pytest 7건 통과
코드 품질	ruff check 오류 없음

1. 실제 실행 결과

세 API 동시 수집 및 Pydantic 검증

```
INFO | API 응답 정상: ip_location (200)
INFO | API 응답 정상: country (200)
INFO | API 응답 정상: weather (200)
INFO | 스키마 검증 성공: weather (72건)
INFO | 스키마 검증 성공: country (1건)
INFO | 스키마 검증 성공: ip_location (1건)
```

=== 수집 및 검증 결과 ===

```
weather: 72건
country: 1건
ip_location: 1건
```

CSV와 Parquet 저장 성능

=== CSV 및 Parquet 성능 비교 ===

```
CSV    | 쓰기 0.005322초 | 읽기 0.004915초 | 크기 3,238 bytes
PARQUET | 쓰기 2.089401초 | 읽기 3.521477초 | 크기 12,144 bytes
```

재로딩 검증: CSV 74건 / Parquet 74건 일치

pytest 및 Ruff

```
===== test session starts =====
collected 7 items

7 passed in 2.65s

$ ruff check .
All checks passed!
```

2. 코드 분석 및 본인 의견

비동기 수집

API 요청은 네트워크 응답을 기다리는 I/O 작업이다. 세 요청을 순서대로 기다리지 않고 `asyncio.gather()`로 동시에 진행하여 전체 대기 시간을 줄였다.

스키마 분리

날씨, 국가, IP 응답은 구조가 서로 다르므로 Pydantic 모델을 세 개로 분리했다. 강수확률 0~100, 양수 인구, 위도와 경도 범위 같은 규칙을 필드 가까이 명시했다.

저장 형식 비교

이번 결과는 74건으로 매우 작다. CSV는 준비 비용과 파일 크기 면에서 유리했고, Parquet은 엔진 초기화와 메타데이터 비용 때문에 더 느리고 컸다. 컬럼형 저장의 장점이 나타나기에는 데이터가 너무 작다는 의미다.

오류 처리

HTTP 상태 오류, 응답 구조 오류, Pydantic `ValidationError`를 구분해 출력한다. 일부 API가 실패했는데도 성공한 것처럼 저장하지 않도록 비정상 종료한다.

3. 개선 사항

- 일시적인 네트워크 오류에 지수 백오프 재시도 적용
- 마지막 정상 응답을 로컬 캐시에 저장하여 API 장애 대응
- 반복 측정과 워밍업을 적용해 성능 비교의 변동성 감소
- GitHub Actions에서 pytest와 Ruff 자동 실행

결론

교재에서 학습한 비동기 I/O, Pydantic v2, 파일 I/O, 예외 처리, pytest, Ruff를 하나의 재현 가능한 데이터 수집 파이프라인으로 통합했다. 실제 API 세 곳의 응답을 모두 확인했고 저장 후 재로딩 건수도 일치했다.